

# University of Wah

Department of Computer Science



**Submit to:**

Ms. Muniba Khan

**Submit by:**

Muhammad Talha

**Course:**

Data Structure and Algorithm Lab

**Reg No:**

UW-24-CS-BS-101

**Section:**

BS-CS-3rd-B

## Task # 01

### Priority Queue in Ascending Order:

```
#include <iostream>
using namespace std;

class Node {
public:
    int priority;
    int data;
    Node* next;
};

Node* front = NULL;

void insert(int item, int priority) {
    Node* newNode = new Node();
    newNode->data = item;
    newNode->priority = priority;
    newNode->next = NULL;

    if (front == NULL || priority < front->priority) {
        newNode->next = front;
        front = newNode;
    } else {
        Node* temp = front;
        while (temp->next != NULL && temp->next->priority <= priority) {
            temp = temp->next;
        }
        newNode->next = temp->next;
        temp->next = newNode;
    }
}
```

```
void dequeue() {
    if (front == NULL) {
        cout << "Queue Underflow." << endl;
    } else {
        Node* temp = front;
        cout << "Deleted item is: " << temp->data << endl;
        front = front->next;
        delete temp;
    }
}
```

```

void display() {
    if (front == NULL) {
        cout << "Queue is empty." << endl;
    } else {
        Node* temp = front;
        cout << "Priority Data" << endl;
        while (temp != NULL) {
            cout << temp->priority << "          " << temp->data << endl;
            temp = temp->next;
        }
    }
}

int main() {
    int choice, item, priority;

    while (choice != 4) {
        cout << "\n--- Priority Queue Menu ---\n";
        cout << "1. Insert\n";
        cout << "2. Delete\n";
        cout << "3. Display\n";
        cout << "4. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;
    }
}

```

```

        switch (choice) {
            case 1:
                cout << "Enter data: ";
                cin >> item;
                cout << "Enter priority: ";
                cin >> priority;
                insert(item, priority);
                break;

            case 2:
                dequeue();
                break;

            case 3:
                display();
                break;

            case 4:
                cout << "Exiting program." << endl;
                break;

            default:
                cout << "Invalid choice!" << endl;
        }
    }

    return 0;
}

```

## Output

```
--- Priority Queue Menu ---
1. Insert
2. Delete
3. Display
4. Exit
Enter your choice: 1
Enter data: 32
Enter priority: 3

--- Priority Queue Menu ---
1. Insert
2. Delete
3. Display
4. Exit
Enter your choice: 1
Enter data: 43
Enter priority: 1

--- Priority Queue Menu ---
1. Insert
2. Delete
3. Display
4. Exit
Enter your choice: 3
Priority  Data
1       43
3       32
```

## Task # 02

### Priority Queue in Decending Order:

```

class Node {
public:
    int priority;
    int data;
    Node* next;
};

Node* front = NULL;

void insert(int item, int priority) {
    Node* newNode = new Node();
    newNode->data = item;
    newNode->priority = priority;
    newNode->next = NULL;

    // Descending order: higher priority first
    if (front == NULL || priority > front->priority) {
        newNode->next = front;
        front = newNode;
    } else {
        Node* temp = front;
        while (temp->next != NULL && temp->next->priority >= priority) {
            temp = temp->next;
        }
        newNode->next = temp->next;
        temp->next = newNode;
    }
}

```

```

void dequeue() {
    if (front == NULL) {
        cout << "Queue Underflow." << endl;
    } else {
        Node* temp = front;
        cout << "Deleted item is: " << temp->data
              << " (Priority " << temp->priority << ")" << endl;
        front = front->next;
        delete temp;
    }
}

void display() {
    if (front == NULL) {
        cout << "Queue is empty." << endl;
    } else {
        Node* temp = front;
        cout << "Priority  Data" << endl;
        while (temp != NULL) {
            cout << temp->priority << "          " << temp->data << endl;
            temp = temp->next;
        }
    }
}

```

```

int main() {
    int choice, item, priority;

    do {
        cout << "\n--- Priority Queue (Descending Order) ---\n";
        cout << "1. Insert\n";
        cout << "2. Delete\n";
        cout << "3. Display\n";
        cout << "4. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice) {
            case 1:
                cout << "Enter data: ";
                cin >> item;
                cout << "Enter priority: ";
                cin >> priority;
                insert(item, priority);
                break;

            case 2:
                dequeue();
                break;

            case 3:
                display();
                break;

            case 4:
                cout << "Exiting program." << endl;

```

## Output

```

--- Priority Queue (Descending Order) ---
1. Insert
2. Delete
3. Display
4. Exit
Enter your choice: 1
Enter data: 54
Enter priority: 22

--- Priority Queue (Descending Order) ---
1. Insert
2. Delete
3. Display
4. Exit
Enter your choice: 1
Enter data: 43
Enter priority: 3

--- Priority Queue (Descending Order) ---
1. Insert
2. Delete
3. Display
4. Exit
Enter your choice: 3
Priority  Data
23      4
22      54
3       43

```

### Task # 03

#### CPU Scheduling

CPU Scheduling is the method used by the operating system to decide which process gets the CPU and for how long.

When many programs want to run, CPU scheduling decides who runs first.

#### **Example:**

- CPU cannot run all at once
- OS schedules them one by one

#### Dijkstra's Algorithm

Dijkstra's Algorithm is used to find the shortest path from one node to all other nodes in a graph.

It finds the minimum distance between places.

#### **Example:**

- Google Maps finding the shortest route from Home → College
- Network routing (shortest data path)

## Event-Driven Simulation

Event-Driven Simulation is a simulation where the system changes only when an event occurs.

Nothing happens until an event triggers a change.

### Example:

- Bank simulation:
  - Event 1: Customer arrives
  - Event 2: Teller starts service
  - Event 3: Customer leaves